

Transformer Architectures & Attention Mechanisms

Personal Technical Study Notes & Comprehensive Reference Collection

Knowledge Synthesis: Vaswani et al. (2017), DeepLearning.AI Coursework (Jay Alammar & Maarten Grootendorst), and Sharif ML Lectures

Overview & Study Scope

This document serves as a personal reference repository consolidating key mathematical formulations, structural diagrams, and PyTorch implementations of Transformer models. It bridges the gap between high-level intuition (coreference resolution, token tracks) and low-level computational mechanics (matrix multiplications, scaled variance proofs, and dynamic learning rate scheduling).

1 Module 1: High-Level Architecture & Sequential Bottlenecks

Prior to the introduction of Transformers, sequence transduction models relied almost exclusively on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) units.

1.1 The Recurrent Bottleneck

Recurrent models compute a sequence of hidden states h_t as a function of the previous hidden state h_{t-1} and the input at position t :

$$h_t = f(h_{t-1}, x_t) \quad (1)$$

This inherently sequential formulation creates a major computational bottleneck: computation at step t cannot begin until step $t - 1$ completes. This precludes parallelization within training examples across GPU hardware and restricts maximum context windows.

1.2 The Transformer Alternative

The Transformer eschews recurrence entirely, relying on global self-attention to connect all tokens in a sequence in $O(1)$ sequential operations. Tokens flow as continuous vectors of dimension $d_{\text{model}} = 512$ through a stack of $N = 6$ identical encoder and decoder blocks.

Each sub-layer output undergoes residual addition followed by layer normalization:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (2)$$

—

2 Module 2: Scaled Dot-Product Attention: Mechanics & Variance Proof

An attention function maps a set of Queries (Q), Keys (K), and Values (V) to an output vector space.

2.1 Mathematical Formulation

The core matrix operation for Scaled Dot-Product Attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, and $V \in \mathbb{R}^{m \times d_v}$.

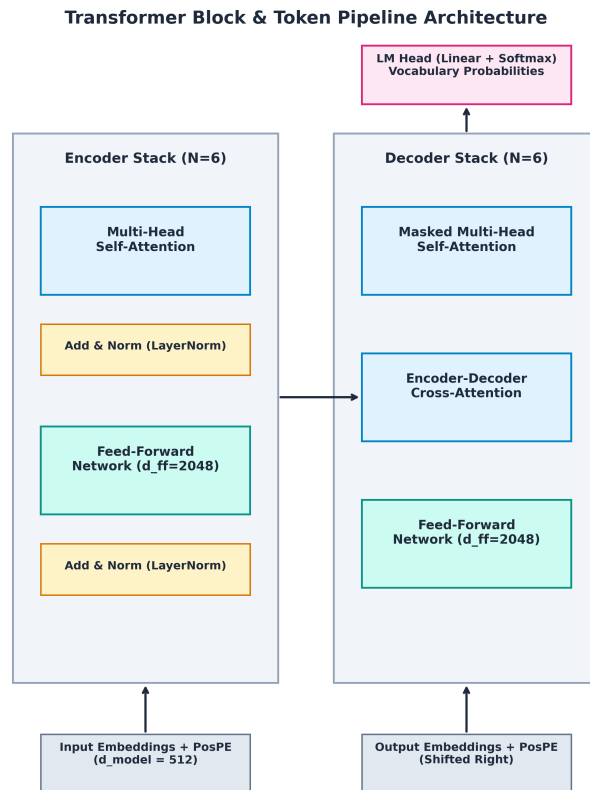


Figure 1: **Complete Transformer Architecture Layout.** Flow of token embeddings through stacked Encoder and Decoder layers, featuring Multi-Head Attention, Feed-Forward sub-layers, Residual Connections, and the final LM Head.

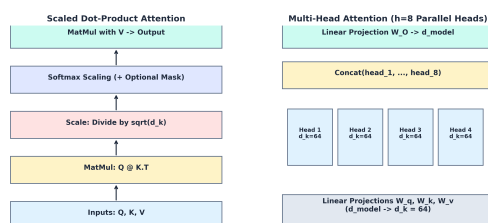


Figure 2: **Scaled Dot-Product Attention vs. Multi-Head Attention** execution pipelines.

2.2 Mathematical Proof: Scaling Factor $\frac{1}{\sqrt{d_k}}$

Why divide by $\sqrt{d_k}$? Consider two independent random vectors $q, k \in \mathbb{R}^{d_k}$ with components having mean 0 and variance 1:

$$\mathbb{E}[q_i] = 0, \quad \text{Var}(q_i) = 1, \quad \mathbb{E}[k_i] = 0, \quad \text{Var}(k_i) = 1 \quad (4)$$

The dot product is $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$. The mean of the sum is 0, and the variance is:

$$\text{Var}(q \cdot k) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = \sum_{i=1}^{d_k} (1)(1) = d_k \quad (5)$$

For large key dimensions ($d_k = 64$ or larger), the variance of the dot product scales up to d_k , pushing dot products into extreme magnitudes. High-magnitude inputs push the Softmax function into regions with near-zero gradients. Dividing by $\sqrt{d_k}$ scales the variance back down to $\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = 1$, stabilizing training gradients.

3 Module 3: Multi-Head Attention & Representation Subspaces

Instead of computing a single attention pass with $d_{\text{model}} = 512$, Multi-Head Attention projects Queries, Keys, and Values h times with learned linear projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (6)$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (7)$$

Parameter Projection Matrices:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, \quad W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}} \quad (8)$$

For the base model: $h = 8$, $d_{\text{model}} = 512$, and $d_k = d_v = d_{\text{model}}/h = 64$.

4 Module 4: Coreference Resolution & Contextual Embedding Baking

A key conceptual insight from DeepLearning.AI is **Contextual Embedding Baking** and **Coreference Resolution**.

In the prompt **"The dog chased the llama because it..."**, self-attention allows the model to determine what **"it"** refers to. As illustrated in Figure 3, relevance scoring assigns high weight (0.72) to **"llama"**. The weighted combination step then multiplies this score by the value vector V_{llama} , effectively **"baking"** the representation of **"llama"** into the current vector for **"it"**.

5 Module 5: Position-wise Feed-Forward Networks (FFN)

Each Transformer block contains a fully connected feed-forward sub-layer applied to each position separately and identically:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (9)$$

While the input and output dimensions remain $d_{\text{model}} = 512$, the inner-layer expands to $d_{\text{ff}} = 2048$. This high-dimensional expansion allows the network to memorize statistical associations and world knowledge (e.g., predicting **"redemption"** after **"Shawshank"**).

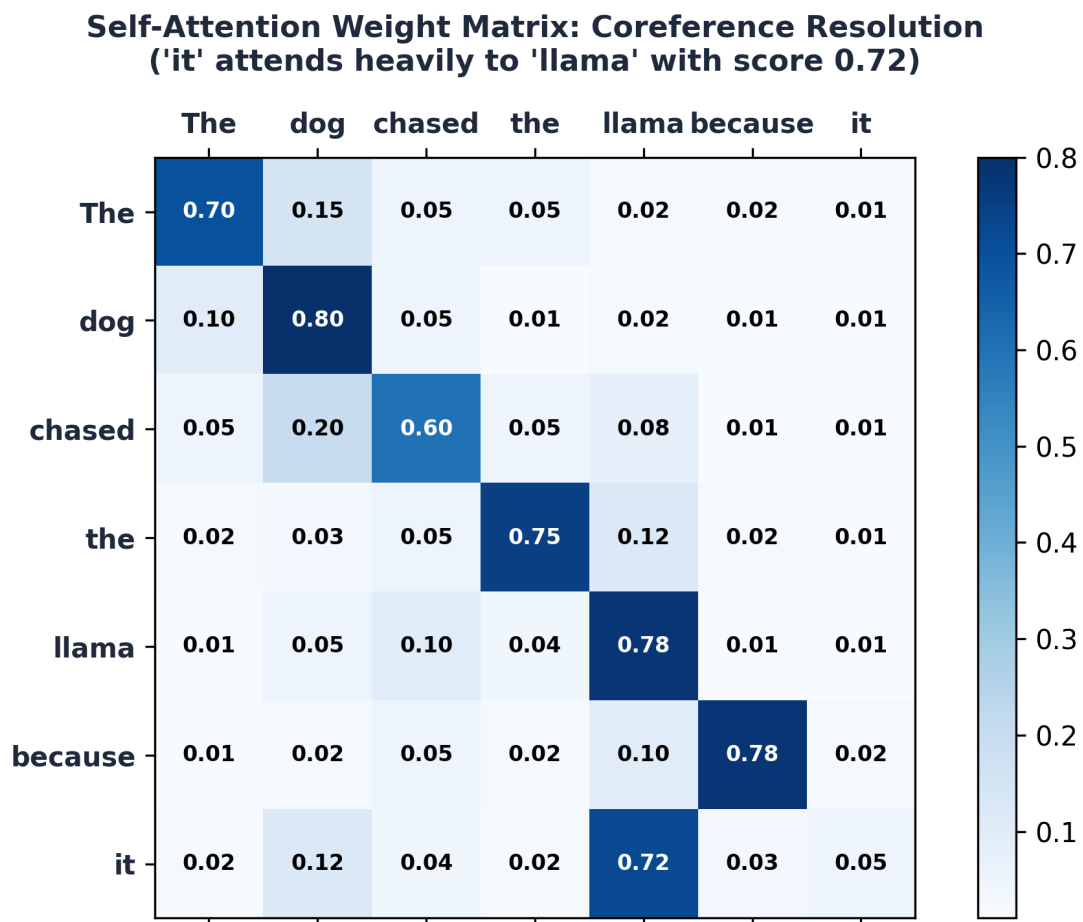


Figure 3: **Self-Attention Coreference Resolution & Embedding Baking.** How self-attention calculates relevance scores to bake context from antecedent tokens into ambiguous pronouns.

6 Module 6: Sinusoidal Positional Encodings

To inject sequence order without recurrence, fixed sinusoidal positional encodings are added to input embeddings:

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (10)$$

$$\text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (11)$$

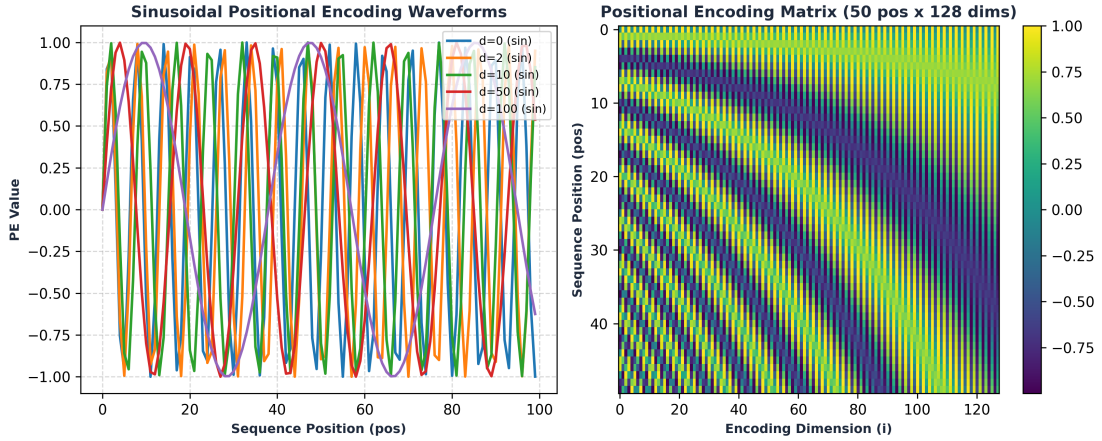


Figure 4: **Sinusoidal Positional Encoding Curves & Heatmap Matrix.** Geometric frequency progressions spanning dimensions $0 \leq 2i < 512$.

7 Module 7: Theoretical Complexity & Optimization Schedule

7.1 Computational Complexity Matrix

Table 1 summarizes the per-layer resource requirements comparing Self-Attention against Recurrent and Convolutional alternatives.

Table 1: Per-Layer Computational Complexity, Sequential Operations, and Maximum Path Lengths (n : sequence length, d : representation dimension, k : kernel size).

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Restricted Self-Attention	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

7.2 Adam Optimizer & Learning Rate Schedule

Models are trained using Adam ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$) with a dynamic learning rate schedule:

$$\text{lrate} = d_{\text{model}}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5}) \quad (12)$$

with $\text{warmup_steps} = 4000$.

8 Module 8: Complete PyTorch Reference Implementation

Below is my reference PyTorch code implementation reflecting the architectural modules documented above:

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class ScaledDotProductAttention(nn.Module):
7     def __init__(self):
8         super().__init__()
9
10    def forward(self, Q, K, V, mask=None):
11        d_k = Q.size(-1)
12        # Scaled dot-product calculation
13        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
14
15        if mask is not None:
16            scores = scores.masked_fill(mask == 0, -1e9)
17
18        attn_weights = F.softmax(scores, dim=-1)
19        output = torch.matmul(attn_weights, V)
20        return output, attn_weights
21
22 class MultiHeadAttention(nn.Module):
23     def __init__(self, d_model=512, h=8):
24         super().__init__()
25         self.d_model = d_model
26         self.h = h
27         self.d_k = d_model // h # 64
28
29         self.W_q = nn.Linear(d_model, d_model)
30         self.W_k = nn.Linear(d_model, d_model)
31         self.W_v = nn.Linear(d_model, d_model)
32         self.W_o = nn.Linear(d_model, d_model)
33         self.attention = ScaledDotProductAttention()
34
35     def forward(self, Q, K, V, mask=None):
36         batch_size = Q.size(0)
37
38         # Split into h heads: (batch, seq, h, d_k) -> (batch, h, seq, d_k)
39         q_proj = self.W_q(Q).view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
40         k_proj = self.W_k(K).view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
41         v_proj = self.W_v(V).view(batch_size, -1, self.h, self.d_k).transpose(1, 2)
42
43         out, _ = self.attention(q_proj, k_proj, v_proj, mask=mask)
44
45         # Concatenate heads and project
46         out = out.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)
47         return self.W_o(out)

```

Listing 1: PyTorch Implementation of Scaled Dot-Product Attention and Multi-Head Attention.

```

1 class PositionwiseFeedForward(nn.Module):
2     def __init__(self, d_model=512, d_ff=2048):
3         super().__init__()
4         self.w_1 = nn.Linear(d_model, d_ff)
5         self.w_2 = nn.Linear(d_ff, d_model)
6
7     def forward(self, x):
8         return self.w_2(F.relu(self.w_1(x)))
9
10 class PositionalEncoding(nn.Module):
11     def __init__(self, d_model=512, max_len=5000):
12         super().__init__()

```

```
13     pe = torch.zeros(max_len, d_model)
14     position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
15     div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0)
    / d_model))
16
17     pe[:, 0::2] = torch.sin(position * div_term)
18     pe[:, 1::2] = torch.cos(position * div_term)
19     self.register_buffer('pe', pe.unsqueeze(0))
20
21     def forward(self, x):
22         return x + self.pe[:, :x.size(1)]
23
24 class TransformerEncoderBlock(nn.Module):
25     def __init__(self, d_model=512, h=8, d_ff=2048, dropout=0.1):
26         super().__init__()
27         self.attn = MultiHeadAttention(d_model=d_model, h=h)
28         self.ffn = PositionwiseFeedForward(d_model=d_model, d_ff=d_ff)
29         self.norm1 = nn.LayerNorm(d_model)
30         self.norm2 = nn.LayerNorm(d_model)
31         self.dropout = nn.Dropout(dropout)
32
33     def forward(self, x, mask=None):
34         attn_out = self.attn(x, x, x, mask=mask)
35         x = self.norm1(x + self.dropout(attn_out))
36         ffn_out = self.ffn(x)
37         x = self.norm2(x + self.dropout(ffn_out))
38         return x
```

Listing 2: Positionwise Feed-Forward Network and Positional Encoding Modules.